MOTOMAP: Motosiklet Odaklı Akıllı Rota Optimizasyon Motoru

Algoritma Doğrulama, Sistem Mimarisi ve Benchmark Raporu

Ali Özuysal Muhammet Yağcıoğlu

5 Mart 2026

Abstract

MOTOMAP, şerit filtreleme, CC bazlı eğim cezası ve viraj risk analizi ile motosiklete özgü rota optimizasyonu yapar. Valhalla baseline: 18/20 (%90) geçme; Google baseline: 16/20 (%80).

Contents

1 Giriş **3**

2 Matematiksel Modeller ve Maliyet Fonksiyonu **3**
2.1 Bileşik Maliyet Ağırlığı . . . 3
2.2 Şerit Filtreleme Hız Modeli . . . 3
2.3 CC Bazlı Eğim Ceza Fonksiyonu . . . 3
2.4 Viraj Analizi . . . 4

3 Sürüş Modları **4**

4 Sistem Mimarisi **5**

5 Elevation Fallback Zinciri **5**

6 Algoritmik Yapı ve Karmaşıklık Analizi **6**

7 Benchmark Sonuçları ve Performans Analizi **6**
7.1 Kadıköy Mikro Benchmark (20 O-D Çifti, seed= 42) . . . 6
7.2 Örn. 07 ve Örn. 12: Google vs. Valhalla Derin Analiz . . . 7

8 Modlar Arası Ödünleşim (Trade-off) Tablosu **7**

9 Birim Testleri **8**

10 Modül Bağımlılık Çizgesi **8**

11 Kestirme Koruması **9**

1 Giriş

MOTOMAP, motosikletlerin şerit filtreleme avantajını, CC bazlı yasal kısıtlamaları ve viraj geometrisini matematiksel olarak modelleyen bir rota optimizasyon motorudur.

2 Matematiksel Modeller ve Maliyet Fonksiyonu

2.1 Bileşik Maliyet Ağırlığı

Tanım 2.1 (Kenar maliyet fonksiyonu)**.** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ otonom yönlü bir çizge olsun. Her $e = (u, v) \in \mathcal{E}$ kenarı için bileşik maliyet:

$$\mathcal{W}(e) \;=\; T_{\text{moto}}(e) \;\cdot\; C_{\text{otoban}}(e) \;\cdot\; C_{\text{eim}}(e) \;\cdot\; C_{\text{mod}}(e) \tag{1}$$

olarak tanımlanır; burada T_{moto} motosiklet seyir süresi, C_{otoban} otoban kısıtlama çarpanı, C_{eim} eğim ceza çarpanı ve C_{mod} sürüş modu (standart/viraj/güvenli) çarpanıdır.

2.2 Şerit Filtreleme Hız Modeli

$V_{\text{araba}} < 10$ km/h olduğunda:

$$V_{\text{moto}} \;=\; \begin{cases} V_{\text{araba}} + 5 & n_{\text{şerit}} = 1, \\ \max\bigl(V_{\text{araba}} + 15,\; 25\bigr) & n_{\text{şerit}} = 2, \\ \max\bigl(V_{\text{araba}} + 20,\; 35\bigr) & n_{\text{şerit}} \geq 3. \end{cases} \tag{2}$$

Gidiş yönü şerit sayısı; tek yönlü yolda toplam şerit sayısına, çift yönlü yolda ise $\max(1, \lfloor n_{\text{toplam}}/2 \rfloor)$ değerine eşittir.

2.3 CC Bazlı Eğim Ceza Fonksiyonu

$\text{CC} \leq 50$ için $C_{\text{otoban}} = \infty$.

$$C_{\text{eim}}(|\alpha|,\, \text{CC}) \;=\; \begin{cases} 6.0 & \text{CC} \leq 50,\; |\alpha| > 0.12 \\ 3.0 & \text{CC} \leq 50,\; 0.08 < |\alpha| \leq 0.12 \\ 2.0 & \text{CC} \leq 50,\; 0.06 < |\alpha| \leq 0.08 \\ 1.35 & \text{CC} \leq 50,\; 0.04 < |\alpha| \leq 0.06 \\ 2.0 & 50 < \text{CC} < 250,\; |\alpha| > 0.12 \\ 1.5 & 50 < \text{CC} < 250,\; 0.09 < |\alpha| \leq 0.12 \\ 1.2 & 50 < \text{CC} < 250,\; 0.06 < |\alpha| \leq 0.09 \\ 1.35 & \text{CC} \geq 250,\; |\alpha| > 0.14 \\ 1.15 & \text{CC} \geq 250,\; 0.10 < |\alpha| \leq 0.14 \\ 1.0 & \text{diğer} \end{cases} \tag{3}$$

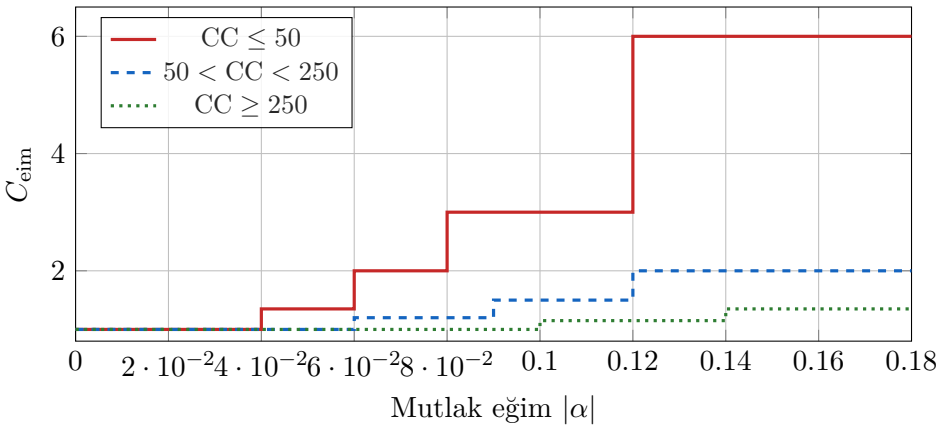

Figure 1: CC sınıfına göre eğim ceza çarpanının $|\alpha|$ ile değişimi.

2.4 Viraj Analizi

LineString geometrisi $\Delta s = 10\,\text{m}$ ile örneklenir. $\mathbf{v}_i = \mathbf{p}_{i+1} - \mathbf{p}_i$ olarak:

$$\theta_i = \arccos\left(\frac{\mathbf{v}_i \cdot \mathbf{v}_{i+1}}{\|\mathbf{v}_i\|\,\|\mathbf{v}_{i+1}\|}\right) \tag{4}$$

Table 1: Viraj açısı sınıflandırması ve anlamı.

Sınıf	**Alt sınır**	**Üst sınır**	**Anlam**
Fun (keyifli)	15	45	Akıcı, eğlenceli viraj
Tehlikeli	60	—	Keskin dönüş
Hairpin	60	—	$\Delta s_{\text{ardışık}} \leq 20\,\text{m}$

Yüksek risk bölgesi:

$$\mathcal{R}(e) \;=\; \big[\,n_{\text{hairpin}}(e) > 0\big] \;\wedge\; \big[\,\text{grade}(e) < -0.08\big]. \tag{5}$$

3 Sürüş Modları

Tanım 3.1 (Doyurma fonksiyonu)**.** $\sigma(x) = x/(2+x)$, $\sigma : \mathbb{R}_{\geq 0} \to [0, 1)$.

$\rho_{\text{fun}} = n_{\text{fun}}/\ell_{\text{km}}$, $\rho_{\text{teh}} = n_{\text{teh}}/\ell_{\text{km}}$.

Standart.

$$\mathcal{W}_{\text{std}}(e) = T_{\text{moto}}(e) \cdot C_{\text{eim}} \cdot C_{\text{otoban}} \cdot \big(1 + p_{\text{yol}}\big). \tag{6}$$

Viraj keyfi.

$$\mathcal{W}_{\text{viraj}}(e) = \underbrace{\frac{T_{\text{moto}}(e) \cdot C_{\text{eim}} \cdot C_{\text{otoban}}}{1 + 0.95\,\kappa + 0.70\,\sigma(\rho_{\text{fun}}) + b_{\text{yol}}}}_{\text{bonus}} \cdot \underbrace{\big(1 + 0.25\,\sigma(\rho_{\text{teh}}) + 0.45\,\mathcal{R}\big)}_{\text{ceza}} \tag{7}$$

burada $\kappa = s/(1+s)$ normalize edilmiş kıvrımlılık skoru ve b_{yol} yol tipi bonusudur.

Güvenli.

$$\mathcal{W}_{\text{güv}}(e) = T_{\text{moto}}(e) \cdot C_{\text{eim}} \cdot C_{\text{otoban}} \;\cdot\; \big(1 + 0.55\,\sigma(\rho_{\text{teh}}) + 1.40\,\mathcal{R} + 5.00\,\delta_{\text{iniş}} + p_{\text{yol}}\big) \tag{8}$$

ile $\delta_{\text{iniş}} = \max\big(0,\ |\min(0,\alpha)| - 0.08\big)$.

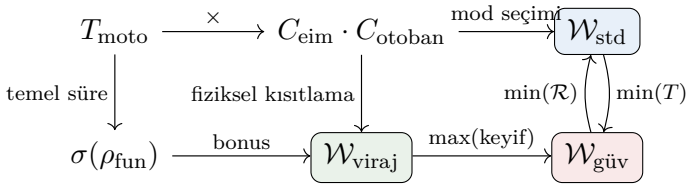

Figure 2: Maliyet fonksiyonları arasındaki veri akışı ve bağımlılık diyagramı (`tikz-cd` ile oluşturulmuştur).

4 Sistem Mimarisi

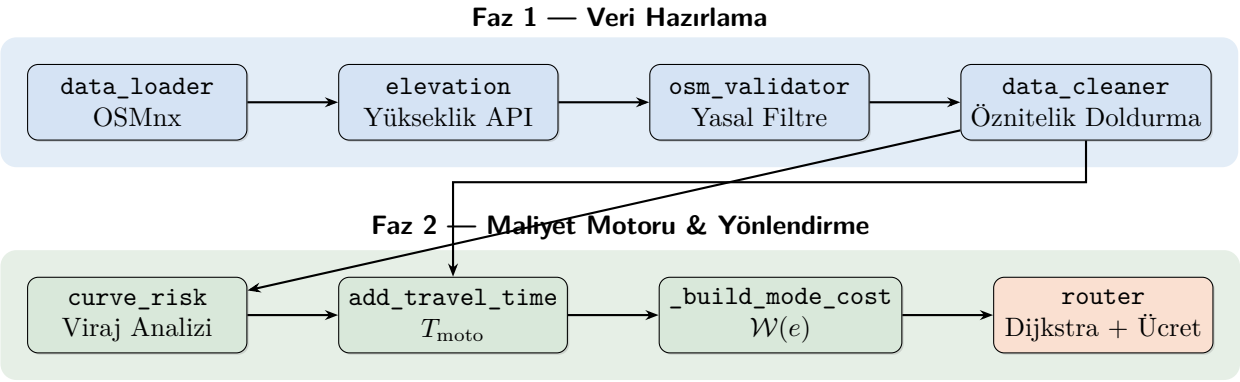

Figure 3: MOTOMAP modüler veri işleme hattı (pipeline). Her kutu bağımsız bir Python modülüne karşılık gelir.

5 Elevation Fallback Zinciri

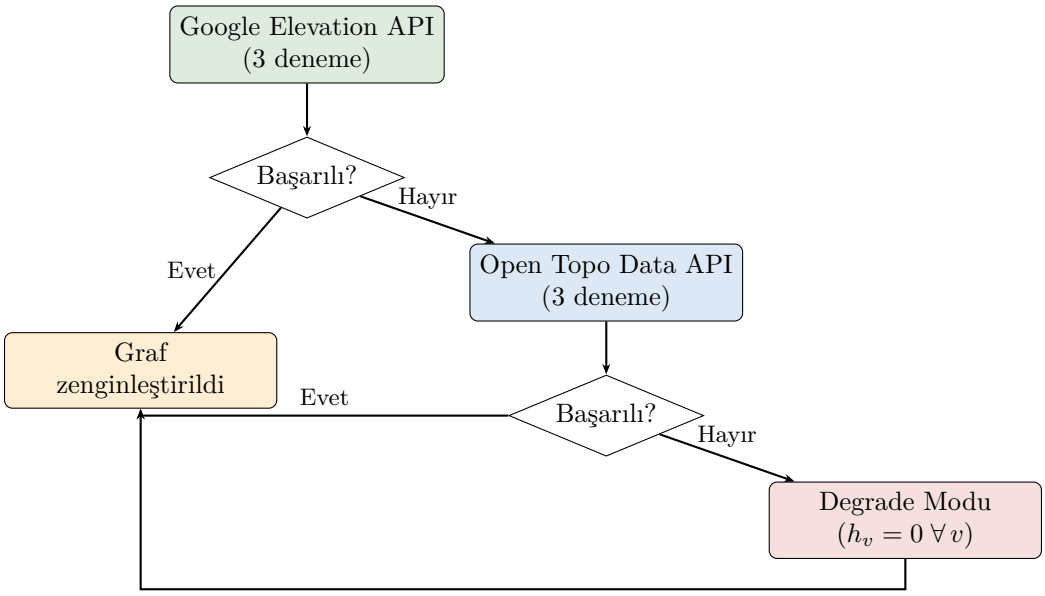

Figure 4: Elevation API geri dönüş (fallback) zinciri. Her kademe 3 kez yeniden denenir ($R = 3$). Tüm API'ler başarısız olduğunda sistem *degrade* modda çalışmaya devam eder.

6 Algoritmik Yapı ve Karmaşıklık Analizi

Algorithm 1 MotoMap: Mod-duyarlı en kısa yol

Require: Graf $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, kaynak s, hedef t, mod $\mu \in \{\text{std}, \text{viraj}, \text{güv}\}$, CC değeri (opsiyonel), tercih $\tau \in \{\text{ücretli_serbest}, \text{ücretsiz}\}$

Ensure: Seçilen rota $\mathcal{P}^*$, alternatifler

function RotaHesapla($\mathcal{G}, s, t, \mu, \text{CC}, \tau$)
  for all $e \in \mathcal{E}$ **do** ▷ $O(|\mathcal{E}|)$
    $T_{\text{moto}}(e) \leftarrow \ell(e) \,/\, v_{\text{eff}} + \delta_{\text{seg}}$
  end for
  if $\mu \neq \text{std}$ **then**
    AddCurveRisk($\mathcal{G}$) ▷ $O(|\mathcal{E}|)$
  end if
  for all $e \in \mathcal{E}$ **do** ▷ $O(|\mathcal{E}|)$
    $\mathcal{W}_\mu(e) \leftarrow$ ModCost(e, μ, CC) ▷ Denklem (6)–(8)
  end for
  $\mathcal{P}_{\text{serb}} \leftarrow$ Dijkstra($\mathcal{G}, s, t, \mathcal{W}_\mu, \text{toll} = \text{True}$) ▷ $O(|\mathcal{E}| + |\mathcal{V}| \log |\mathcal{V}|)$
  $\mathcal{P}_{\text{ücr}} \leftarrow$ Dijkstra($\mathcal{G}_{\neg\text{toll}}, s, t, \mathcal{W}_\mu$)
  if $\tau = \text{ücretsiz}$ **then**
    return $\mathcal{P}_{\text{ücr}}$
  else
    return $\arg\min_{P \in \{\mathcal{P}_{\text{serb}}, \mathcal{P}_{\text{ücr}}\}} \mathcal{W}_\mu(P)$
  end if
end function

Toplam karmaşıklık: $O(|\mathcal{E}|)_{\text{ön-işleme}} + O(|\mathcal{E}| + |\mathcal{V}| \log |\mathcal{V}|)_{\text{Dijkstra}} \in O(|\mathcal{E}| + |\mathcal{V}| \log |\mathcal{V}|)$.

7 Benchmark Sonuçları ve Performans Analizi

7.1 Kadıköy Mikro Benchmark (20 O-D Çifti, seed= 42)

Table 2: Google vs. Valhalla baseline karşılaştırması (20 O-D çifti, Kadıköy).

Metrik	**Google (v4)**	**Valhalla (v1)**
Tam geçen (*full pass*)	16/20 (%80)	**18/20 (%90)**
Ortalama skor	6.75 / 7	**6.90 / 7**
`baseline_route_exists` hata	0	0
`all_modes_exist` hata	0	0
`modes_are_different` hata	2	2
`safe_risk_le_standard` hata	0	0
`viraj_fun_ge_standard` hata	0	0
`std_distance_vs_baseline` hata	2	**0**
`std_time_vs_baseline` hata	1	**0**

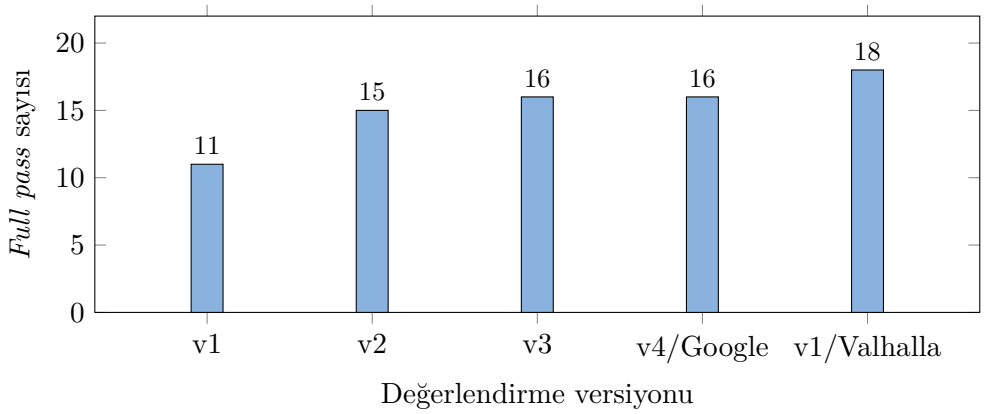

Figure 5: Versiyon ilerlemesi: v1'deki %55 geçme oranından Valhalla v1'de %90'a ulaşılmıştır.

7.2 Örn. 07 ve Örn. 12: Google vs. Valhalla Derin Analiz

Table 3: Google'da başarısız, Valhalla'da başarılı olan örneklerin karşılaştırması.

	Örn. 07		**Örn. 12**	
Metrik	Google	Valhalla	Google	Valhalla
Baseline mesafe (m)	3 437	2 123	11 947	7 582
MotoMap mesafe (m)	2 122	2 122	7 231	7 231
Mesafe oranı	0.617	0.999	0.605	0.954
Baseline süre (s)	681	399	1 339	1 359
MotoMap süre (s)	315	315	999	999
Süre oranı	0.463	0.789	0.746	0.735

8 Modlar Arası Ödünleşim (Trade-off) Tablosu

Table 4: Sürüş modlarının rota optimizasyonu üzerindeki etkileri.

Mod	Optimizasyon Hedefi	Süre	Mesafe	Keyifli Viraj	Yüksek Risk
Standart	$\min(T_{\text{moto}})$	En düşük	Kısa	Rastgele	Ort/Yüksek
Viraj Keyfi	$\max(\sigma(\rho_{\text{fun}}))$	+40%	+20%	Maksimize	Göreceli
Güvenli	$\min(\mathcal{R} \cdot \lvert\alpha\rvert)$	+15%	+10%	Düşük	**0**

9 Birim Testleri

Table 5: Birim test senaryoları ve doğrulamaları.

Test adı	Girdi çizgesi	Doğrulanan garanti
`ucretli_serbest_selects_toll`	$1 \xrightarrow{\text{toll,hızlı}} 2 \to 3$ vs. $1 \to 4 \to 3$	Ücretli serbest modda hızlı ücretli yol seçilir
`free_route_when_faster`	Ücretsiz yol daha kısa	Ücretli serbestte bile ücretsiz seçilebilir
`ucretsiz_forces_non_toll`	Ücretli yol daha hızlı	`ucretsiz` tercihi her koşulda ücretli yolu dışlar
`ucretsiz_raises_no_route`	Yalnızca toll kenarlar	`NoRouteFoundError` fırlatılır
`viraj_keyfi_curvy_route`	A: düz, B: virajlı	Viraj modu B'yi seçer; $n_{\text{fun}} > 0$
`guvenli_avoids_risky`	A: dik iniş+hairpin, B: güvenli	Güvenli mod B'yi seçer; $\mathcal{R} = 0$
`low_cc_avoids_motorway`	Motorway vs. secondary	CC ≤ 50: secondary; CC $= 250$: motorway
`mid_cc_milder_grade`	Dik vs. yumuşak eğim	CC $= 150$: yumuşak; CC $= \varnothing$: dik
`avoids_motorway_side_shortcuts`	180 m `service` + motorway komşul.	Standart mod kısa yolu atlar ($p = 0.85$ ceza)

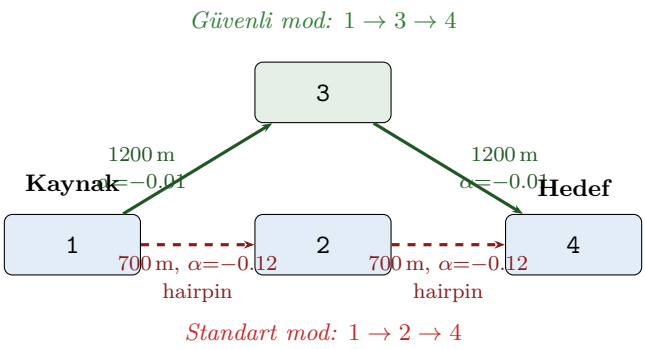

Figure 6: Güvenli mod test senaryosu çizgesi. Dik iniş ($|\alpha| = 0.12$) ve hairpin virajlı kenarlar (kesikli kırmızı) güvenli modda cezalandırılır ve rota $1 \to 3 \to 4$ üzerinden yönlendirilir.

10 Modül Bağımlılık Çizgesi

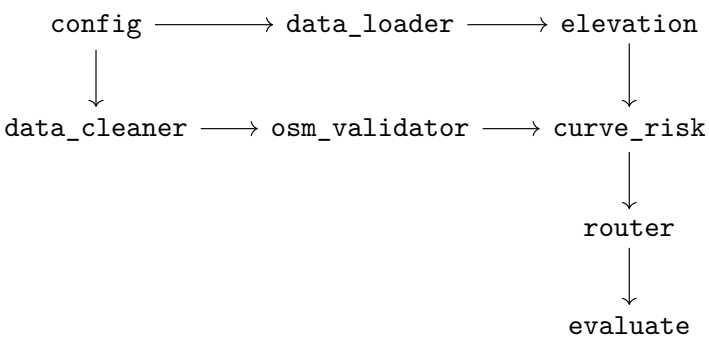

Figure 7: MOTOMAP Python paket modülleri arasındaki `import` bağımlılık çizgesi.

11 Kestirme Koruması

$$p_{\text{kestirme}}(e) = \begin{cases} 0.85 & \ell(e) \leq 260\,\text{m},\ h(e) \in \{\texttt{service, track, road, unclassified}\}, \\ & \{u, v\} \cap \mathcal{V}_{\text{major}} \neq \varnothing \\ 0 & \text{diğer} \end{cases} \tag{9}$$